



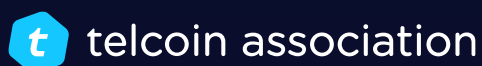
Telcoin Network node guide for mobile network operators

Deploy, sync, stake and operate a node

Version 1.0 · 1 October 2026

support@telcoin.org

<https://github.com/Telcoin-Association/tn-node-deployment>



Contents

1	Introduction	4
1.1	What this guide covers.....	4
1.2	Contact	4
1.3	Vocabulary	5
1.4	Roles are decided on-chain	5
2	Before you start.....	7
2.1	Hardware per role	7
2.2	Operating system.....	7
2.3	Ports	8
2.4	Have these ready	8
3	Install.....	10
3.1	Get the scripts.....	10
3.2	Run setup.....	10
3.3	Install methods	12
3.4	Where things land.....	12
3.5	Back up the keys now	13
3.6	Firewall.....	13
4	Confirm the node is syncing, then synced	14
4.1	Checking by hand	14
4.2	How long it takes	15
5	Day-2 operations	16
5.1	Keep the scripts current.....	16
5.2	Update the node.....	16
5.3	Change the configuration	17
5.4	Service commands.....	17
5.5	Node Manager UI	17
5.6	Testnet add-ons	18
5.7	Remove a node	18
6	Public RPC (https and wss).....	19
6.1	DNS first.....	19
6.2	New install.....	19
6.3	Existing node.....	20
6.4	Verify	20
6.5	Turn it off	20

7	Validators: stake and activate	21
7.1	Prerequisites	21
7.2	Set variables.....	21
7.3	Confirm the ConsensusNFT.....	21
7.4	Export the stake calldata on the node	21
7.5	Read the stake amount.....	22
7.6	Stake	22
7.7	Activate	22
7.8	Watch your status.....	23
7.9	When you get a committee seat	23
7.10	Status values	24
7.11	Rewards and exit.....	24
7.12	What not to do.....	25
8	Automation.....	26
8.1	Setup without prompts	26
8.2	Advertised RPC URL flags.....	27
8.3	Other scripts.....	27
9	Troubleshooting.....	28
9.1	Sync and node mode.....	28
9.2	Setup and preflight	28
9.3	Public RPC and certificates	29
9.4	Service and keys	29
9.5	Staking and exit.....	30
10	Getting help	31

1 Introduction

Telcoin Network is an EVM-compatible layer 1 blockchain. Its validators are GSMA-member mobile network operators, and the Telcoin Association governs the network. Consensus follows the Narwhal and Bullshark protocols, and the execution layer is reth, so every node serves Ethereum-style JSON-RPC. The public testnet is Adiri, chain ID 2017. The setup script lists mainnet as coming soon.

Anyone can run a node. You need no approval to install one, follow consensus and serve JSON-RPC. Validating is a separate, on-chain step. Only GSMA-approved mobile network operators can take it, and only after the Telcoin Association approves them.

The install is the same for everyone. Every node is provisioned validator-capable, and nothing in the install decides whether it validates. What separates a node that follows consensus from one that validates is on-chain state: a stake, an activation and a seat in the committee. The protocol reads that state each epoch and derives the node's role from it.

1.1 What this guide covers

This guide takes one Linux server from a fresh install to a synced Telcoin Network node and keeps it running. For operators the Telcoin Association has approved, it then walks through staking and activation as a validator.

Everything runs through the scripts in the deployment repository, <https://github.com/Telcoin-Association/tn-node-deployment>. You run them on the node itself. The validator steps add Foundry `cast` commands, which you run from the machine that holds your wallet. The repository's [README](#) is the reference for every script option. This guide is the path through them, in the order you need them:

- [Before you start \(page 7\)](#): hardware, operating system, ports and what to have ready.
- [Install \(page 10\)](#): fetch the scripts, run setup, back up the keys and set up the firewall.
- [Confirm the node is syncing, then synced \(page 14\)](#): how to tell a healthy node from a stuck one, and how long sync takes.
- [Day-2 operations \(page 16\)](#): script and node updates, configuration changes, service commands, the Node Manager UI, testnet add-ons and removal.
- [Public RPC \(https and wss\) \(page 19\)](#): serve JSON-RPC and WebSocket on your own DNS name.
- [Validators: stake and activate \(page 21\)](#): staking, activation, committee seats, rewards and exit.
- [Automation \(page 26\)](#): run setup and the other scripts without prompts.
- [Troubleshooting \(page 28\)](#): symptoms, likely causes and fixes.
- [Getting help \(page 31\)](#): what to send when you ask for support.

The first five chapters apply to every operator. The validator chapter applies once the Association has approved you.

1.2 Contact

Email support@telcoin.org with any question about running a node. The same address handles validator onboarding, approval and hardware. If you plan to validate, write before you buy hardware.

1.3 Vocabulary

Term	Meaning
Node	One <code>telcoin-network</code> install on one server, run by the <code>telcoin</code> systemd unit.
Epoch	The period between committee changes. Testnet epochs are six hours long.
Committee	The validators that vote on consensus during one epoch.
Validator	A node whose execution address has staked and been activated in the ConsensusRegistry contract. It votes only while it holds a committee seat.
Observer	Any node that is not in the current committee.
Execution address	The <code>0x</code> address you enter at key generation. It stakes, receives rewards, and is signed into the node's proof of possession.
BLS key	The node's consensus signing key, stored in <code>node-keys/</code> and encrypted with your BLS passphrase.
<code>node-info.yaml</code>	The node's public identity: BLS public key, proof of possession, P2P addresses and any advertised RPC URLs.
ConsensusNFT	A non-transferable token the Association mints to an approved execution address. Staking requires it.

1.4 Roles are decided on-chain

The protocol derives each node's role every epoch from committee membership. A node that is not in the current committee is an observer, whether or not it has staked. There is no setting in the install that makes a node a validator or an observer.

Observers come in two shapes:

- A follower keeps its RPC on `127.0.0.1` and serves only the box it runs on.
- A public-RPC observer also serves `https://` and `wss://` on its own DNS name through Caddy, and advertises those URLs in `node-info.yaml` so wallets and gateways can find it (see [Public RPC \(https and wss\) \(page 19\)](#)).

Becoming a validator takes four steps, in this order:

1. Association approval, which ends with a ConsensusNFT minted to your execution address (see [Prerequisites \(page 21\)](#)).
2. Stake (see [Stake \(page 22\)](#)).
3. Activation (see [Activate \(page 22\)](#)).
4. Committee selection (see [When you get a committee seat \(page 23\)](#)).

The node software does not change between them.

The node reports what it is doing through the `tn_nodeMode` JSON-RPC method:

<code>tn_nodeMode</code>	Meaning
<code>CvvActive</code>	Voting in the current committee.
<code>CvvInactive</code>	A committee member that is catching up before it can vote. This should be brief.
<code>Observer</code>	Not in the current committee. Every follower, every public-RPC observer, and every staked validator between seats reports this.

2 Before you start

2.1 Hardware per role

These figures come from the hardware requirements page in the telcoin-network documentation. CPU counts are physical cores. Cloud vCPUs are usually hyperthreads, so an 8 vCPU instance has about 4 physical cores.

Role	Minimum	Recommended
Observer, follower	2 cores, 8 GB RAM	4 cores, 16 GB RAM
Observer, public RPC	4 cores, 16 GB RAM	8 cores, 32 GB RAM
Validator	8 cores, 32 GB ECC RAM	16 cores, 64 GB ECC RAM

Storage is the same for every role: at least 2 TB of TLC NVMe (10,000 write IOPS, 300 MB/s), 4 TB recommended. Avoid QLC drives; they wear out under continuous chain writes.

Validators have two network requirements on top of that:

- 200 Mbps symmetric bandwidth at minimum, 1 Gbps recommended.
- A p95 round-trip time well below 1 second to at least 7 of the 10 committee members.

IMPORTANT

Email support@telcoin.org with your planned hardware before you buy anything for a validator.

The same documentation page recommends turning swap off and running time sync (chrony or systemd-timesyncd) on every node. A node rejects headers timestamped more than a second ahead of its own clock. The deployment scripts check neither, so set both up yourself.

`setup-node.sh` runs a hardware check during preflight and prints one line per role, for example `[OK] Full node (follower): meets minimum` or `[WARN] Validator (minimum): below minimum`. The check only warns; it never blocks setup. It counts the logical CPUs that `nproc` reports, so on a cloud instance halve its CPU number before comparing it with the table. It does not check for ECC memory or NVMe.

2.2 Operating system

`setup-node.sh` needs `systemd` 247 or newer, because it protects the BLS passphrase with `systemd LoadCredential`. That means Ubuntu 22.04 or later, Debian 12 or later, or RHEL 9 or later. Older systems stop with `systemd <version> detected -- version 247+ required`.

`setup-node.sh` does not support macOS.

A few parts assume a Debian-family system:

- Building from source installs its build dependencies with `apt`.
- `install-caddy.sh` (public RPC and the public dashboard) installs Caddy from its `apt` repository.
- `ui/install-ui.sh` installs Python with `apt-get` when Python 3 is missing.

Preflight also refuses to continue until the CVE-2026-31431 ("Copy Fail") kernel mitigation is in place. The check passes when a file under `/etc/modprobe.d/`, `/lib/modprobe.d/` or `/run/modprobe.d/` contains `install algif_aead /bin/false` and the `algif_aead` module is not loaded. Apply the mitigation from <https://copy.fail> before you start; you may need to unload the module or reboot. The README has the [detection commands](#).

2.3 Ports

Port	Protocol	Open to	Purpose
49590	UDP	Everywhere, inbound	Primary P2P (QUIC). Every node.
49594	UDP	Everywhere, inbound	Worker P2P (QUIC). Every node.
443	TCP	Everywhere, inbound	Caddy, only when it serves public RPC or the dashboard.
80	TCP	Everywhere, inbound	Caddy certificate issuance and the http-to-https redirect. Recommended with Caddy.
8545	TCP	Loopback only	HTTP JSON-RPC. Never open it.
8546	TCP	Loopback only	WebSocket JSON-RPC. Never open it.
9101	TCP	Loopback only	Prometheus metrics.
43174	TCP	104.155.184.201/32, plus any monitoring hosts you add	Health-check endpoint, a testnet add-on. The node listens on it only when you enable the health monitor.
22 (or your SSH port)	TCP	Your admin IPs	SSH.

Open the two UDP ports on every node, including nodes that may never validate. A node that stakes later behind a closed port cannot reach the committee and misses consensus without any obvious error. The UDP ports you open must match the ports in the `multiaddrs` you advertise at key generation.

Open them in every layer that filters traffic: `ufw` on the host (see [Firewall \(page 13\)](#)), the cloud provider's firewall or security group, and the router's port forwards if the server is behind NAT.

2.4 Have these ready

- The execution address (`0x...`) the node will be registered under. For a validator this is the address that stakes and receives rewards, so it should live in a hardware wallet. It is signed into the proof of possession at key generation; changing it later needs `keytool generate pop` and a new `node-info.yaml`.
- A BLS passphrase, stored somewhere offline. The node needs it on every start.
- The server's public IP, and whether it sits behind NAT. Setup detects the public IP with `api.ipify.org` and the internal IP from the network interface, and builds the default `multiaddrs` from them. Correct them at the prompts if they are wrong.
- A mounted data drive with an `/etc/fstab` entry, if the chain data will not live on the boot disk.

- A DNS A record for the node's public name, if you want public RPC (see [Public RPC \(https and wss\)](#) (page 19)).
- Somewhere off the server to keep a backup of the node keys (see [Back up the keys now](#) (page 13)).
- For validators: an approval request with the Association in progress (support@telcoin.org), and a machine with Foundry's `cast` and your wallet.

3 Install

3.1 Get the scripts

Run the installer as your normal user, without `sudo`, so the scripts land in your own home directory at `~/telcoin-node-scripts`:

```
curl -fsSL https://install.telcoin.network | bash
```

If `install.telcoin.network` is unreachable, the raw GitHub URL is a drop-in replacement:

```
URL=https://raw.githubusercontent.com/Telcoin-Association/tn-node-deployment/main  
curl -fsSL "$URL/install.sh" | bash
```

Or clone the repository yourself:

```
git clone https://github.com/Telcoin-Association/tn-node-deployment.git \  
~/telcoin-node-scripts
```

If `~/telcoin-node-scripts` already exists, the installer asks `Re-install and overwrite? [y/N]`. To pick up newer scripts on an existing install, run `bash ~/telcoin-node-scripts/update-scripts.sh` instead (see [Keep the scripts current \(page 16\)](#)).

3.2 Run setup

```
sudo bash ~/telcoin-node-scripts/setup-node.sh
```

Setup is interactive and runs in the order below. Most prompts show their default in brackets; press Enter to accept it. Yes/no prompts default to No.

1. Welcome. Answer `y` to `Ready to begin node setup?`.
2. Preflight ("Step 1 of 8"). Setup checks that it runs as root, detects the distribution and runs the CVE-2026-31431 gate. It lists your mounted filesystems and physical disks, then asks `Data directory [/var/lib/telcoin]`. Enter a path on your data drive if you have one. It then runs the hardware check, an internet check and a port check (in-use ports only warn), installs `curl` and `git` if missing, and checks for `systemd 247` or newer.
3. Network. Choose `1) Adiri Testnet (Chain ID: 2017)`. Mainnet is listed as coming soon and returns you to the menu. Devnet is not offered interactively.
4. Testnet add-ons (testnet only). Each is off by default: the health-monitor endpoint on TCP 43174, log shipping, metrics shipping, a region label if you turned either shipping option on, and VPN admin SSH. Read the [testnet add-ons guide](#) before you opt in.

5. Install method (see [Install methods \(page 12\)](#)). Source builds first ask `Install missing packages now?` if build dependencies are missing (answer y; No exits), then show a version picker whose default on testnet is the latest `-adiri` release tag. Docker installs ask for the image, defaulting to the newest `-adiri` tag. "I already have it" first offers a binary it finds (`Use this binary?`) and asks `Full path to telcoin binary:` only if it finds none or you decline.
6. BLS passphrase protection (not asked for Docker). This menu comes after the source build (20 to 40 minutes) or the Docker install and image pull. For source builds the passphrase question comes only after the build finishes, so stay attached to the session. 1) `systemd LoadCredential` suits most operators. 2) `TPM/vTPM sealing` ties the passphrase to this machine's TPM; it falls back to `LoadCredential` if no TPM is found. Type 1 or 2; Enter alone is not accepted.
7. Node configuration ("Step 3 of 8"). First, RPC access: 1) `Private` keeps RPC on `127.0.0.1`; 2) `Public` asks for the node's DNS name, then always asks for an optional inbound public IP (Enter auto-detects; set it behind NAT; see [Public RPC \(https and wss\) \(page 19\)](#)). Then the ports: P2P primary `49590`, P2P worker `49594`, RPC `8545`, metrics `9101`. Then `Use these default paths?` for the config, log and install directories, and an optional advertised node name.
8. System infrastructure ("Step 4 of 8"). Service user and group, both `telcoin` by default. The user gets no login shell. Docker installs give it UID `1101`. Docker and existing installs may be asked `Clone the repository now to get the chain-config files?`. Answer y, because No means copying the chain configs by hand before step 10.
9. Keys ("Step 5 of 8"). If `node-keys/` already exists, setup asks `Overwrite existing keys?`. On a re-run, answer No (see [What not to do \(page 25\)](#)). Otherwise it asks for the execution address and prints the detected public IP (it asks for one only if detection fails). It then asks for the external primary and worker addresses and, after printing the detected internal IP, the listener primary and worker addresses. Each shows a default built from the detected IP, such as `/ip4/<public-ip>/udp/49590/quic-v1`; check the IP in them. Then it asks for the BLS passphrase twice. The keytool writes the keys and `node-info.yaml`, setup prints `node-info.yaml` and the on-chain next steps, and it waits at `Press Enter to confirm you have backed up your keys`. Do the backup in [Back up the keys now \(page 13\)](#) before you press Enter.
10. Configuration ("Step 6 of 8"). Copies `genesis.yaml`, `committee.yaml` and `parameters.yaml` into the data directory. If it cannot find them, it prints where to get them and waits for you to copy them in.
11. Service ("Step 7 of 8"). Writes the start wrapper, the systemd unit and `/etc/telcoin/.node-meta`, then asks `Start the node now?`. Only if you answer yes and the service comes up does it ask `Enable auto-start on server reboot?`. Answer yes to both.
12. Public RPC, only if you gave a domain and the node started. Setup checks DNS and enables the endpoint. If DNS is not ready yet, setup still finishes and prints the exact command to run later.
13. The testnet add-ons you opted into are applied. Log or metrics shipping asks for your ingest token (hidden; leave it blank to skip).
14. A summary with the paths, ports and next steps.

Setup has no `--help`, and it ignores arguments it does not recognise without a warning, so check the spelling of any flag you pass. [Public RPC \(https and wss\) \(page 19\)](#) covers the public RPC flags, and [Automation \(page 26\)](#) has the full list.

3.3 Install methods

Menu choice	--install-method	What it does	How it updates
1) Build from source	source	Installs Rust and the build dependencies, clones telcoin-network to /opt/telcoin-source, builds it (20 to 40 minutes; log in /tmp/tn-build.log) and installs /opt/telcoin/telcoin-network.	update-node.sh
3) Docker	docker	Installs Docker if missing and pulls the image from us-docker.pkg.dev/telcoin-network/tn-public/adir. The default tag is the newest -adir tag in the registry, or v0.15.0-adir if the registry cannot be reached.	update-node.sh
4) I already have it	existing	Uses a telcoin-network binary you supply.	By hand: replace the binary and restart the service.
2) Pre-built binary	none	Not available yet; the menu sends you back.	

The README has more on [install options](#) and [network binding](#).

3.4 Where things land

Path	What it is
/opt/telcoin/telcoin-network	The node binary for source builds. An existing install runs the binary from the path you gave setup (Binary= in its summary).
/opt/telcoin/start-telcoin.sh	Start wrapper. It reads the BLS passphrase and holds the node's launch command and flags.
/etc/systemd/system/telcoin.service	The systemd unit, telcoin. Docker installs also name the container telcoin.
/var/lib/telcoin/node-keys/	BLS and P2P keys. Back these up.
/var/lib/telcoin/node-info.yaml	Public identity and advertised URLs.
/var/lib/telcoin/genesis/, /var/lib/telcoin/parameters.yaml	Chain configuration.
/var/lib/telcoin/db/	Chain database. This is what grows.
/etc/telcoin/bls-passphrase	BLS passphrase, mode 600, owned by the service user.
/etc/telcoin/.node-meta	Install settings the other scripts read. Root-only (mode 600), so run the scripts with sudo.
/var/log/telcoin/telcoin.log, telcoin-error.log	Node output.
/opt/telcoin-source/	The telcoin-network source tree (source builds).

If you chose a different data directory, read it in place of `/var/lib/telcoin` everywhere in this guide. The README's [system layout](#) shows the full tree.

3.5 Back up the keys now

WARNING

Losing `node-keys/` means losing the node's identity. If you have not staked yet, you can regenerate keys and carry on. Once you have staked, you have to go back to the Association.

Back up `node-keys/` and `node-info.yaml` together. Keep the BLS passphrase somewhere else (a password manager, or paper in a safe), never next to the archive. In a second terminal on the server:

```
(umask 077; sudo tar -C /var/lib/telcoin -czf - node-keys node-info.yaml \  
> ~/telcoin-keys.tgz)
```

Copy the archive off the server from your own machine, then delete it from the server:

```
scp user@SERVER_IP:telcoin-keys.tgz .  
ssh user@SERVER_IP 'shred -u ~/telcoin-keys.tgz'
```

Treat the archive as a secret. Then go back to setup and press Enter.

3.6 Firewall

```
sudo bash ~/telcoin-node-scripts/firewall-setup.sh
```

Choose **2) Enable firewall with recommended defaults**. It sets inbound to deny and outbound to allow, allows SSH, allows TCP 43174 from the Association monitor only, and allows UDP 49590 and 49594 when a node is installed. It asks whether to reset to a clean slate first; No keeps your existing rules.

SSH stays open to the whole internet after this. To restrict it, use **5) Manage trusted IP whitelist** to add your own IP or range first, then remove the blanket SSH rule from the same menu. Adding a whitelist entry does not remove the blanket rule by itself. Test SSH from a new terminal before you close the current one.

`ufw` is only the host layer. Open UDP 49590 and 49594 in the cloud firewall or security group as well, and forward them on the router if the server sits behind NAT. The README covers the rest of the [firewall script](#).

4 Confirm the node is syncing, then synced

Check that the service is up and watch its log for a minute:

```
systemctl status telcoin
journalctl -u telcoin -f
```

Then run the health check:

```
sudo bash ~/telcoin-node-scripts/check-node.sh
```

It compares your node with the network's public RPC (<https://rpc.telcoin.network>) and ends with one verdict:

- `CATCHING UP: <N> execution blocks behind` while the node is more than 50 blocks behind the network (the script's `EVM_SYNC_THRESHOLD`).
- `All checks passed -- node is healthy and caught up (EVM lag <N>)` once it has caught up.
- `<N> issue(s) found -- review warnings/errors above` when something else is wrong.

Run it twice a few minutes apart. The second run also reports how far the local block advanced since the first, which tells you whether the gap is closing. When the node binary supports `tn_nodeMode`, the report also prints a `Consensus role:` line. The script always exits 0, so anything that consumes it should read the verdict line, not the exit code.

4.1 Checking by hand

Do not use `eth_syncing`. On Telcoin Network it returns `false` even while the node is catching up. Compare `eth_blockNumber` on your node with the public RPC instead:

```
bn() { curl -s -X POST -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_blockNumber","params":[]}' "$1" \
  | sed -E 's/.*"result":"(0x[0-9a-fA-F]+)".*/\1/'; }
echo "local:    $(( $(bn http://127.0.0.1:8545) ))"
echo "network:  $(( $(bn https://rpc.telcoin.network) ))"
```

The node is synced when the two numbers stay within a few blocks of each other.

To see the node's consensus role:

```
curl -s -X POST -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"tn_nodeMode","params":[]}' \
  http://127.0.0.1:8545
```

Any node outside the committee returns `"Observer"`, including a staked validator that has not been seated yet.

4.2 How long it takes

There is no published figure for a first sync; it depends on chain height, disk and bandwidth. Watch the lag shrink rather than waiting for a fixed time. Each restart replays recent consensus output before the node catches up again, so a node that is restarted often stays behind. If the lag does not shrink between runs, see [Troubleshooting \(page 28\)](#).

5 Day-2 operations

5.1 Keep the scripts current

```
bash ~/telcoin-node-scripts/update-scripts.sh
```

Run it as your normal user, without `sudo`. It compares every tracked file with the latest version on GitHub, asks `Download and install all updates? [Y/n]`, and checks each download against its published SHA-256 before installing it. If the Node Manager UI is installed and has changed, it redeploys the UI too and may ask for your `sudo` password.

Run it before every node update. `update-node.sh` learns how to handle new releases through these updates.

5.2 Update the node

```
sudo bash ~/telcoin-node-scripts/update-node.sh
```

The script works out whether the node was built from source or runs in Docker and offers the matching versions. Source installs get the same version picker as setup, defaulting to the latest `-adiri` tag. Docker installs get the recent registry tags, newest first, with the latest and current ones marked. Then it asks how far to go:

- 1) `Prepare only` builds the binary or pulls the image while the node keeps running. Apply it later in a quiet window.
- 2) `Prepare AND apply` also stops the node, swaps in the new version, restarts it and checks its health.
- 3) `Cancel`, or any other input, cancels.

A prepared update survives between runs. The next run offers 1) `Apply now`, 2) `Discard and prepare a different update` or 3) `Exit`. `sudo bash ~/telcoin-node-scripts/update-node.sh --discard` drops a prepared update after asking `Discard this pending update?`.

What else to expect:

- On a node registered on-chain as a validator (status 1 to 4), or one whose status cannot be read, the script shows a downtime warning and asks you to type `CONFIRM` before it stops the node.
- After the restart it waits up to 45 seconds for the service to be active and for the node to answer `tn_latestConsensusHeader`. When many nodes restart at once, quorum takes longer to form. Raise the window with `TN_UPDATE_VERIFY_TIMEOUT`, in seconds, for example `sudo TN_UPDATE_VERIFY_TIMEOUT=120 bash ~/telcoin-node-scripts/update-node.sh`.
- If that check fails, it offers to roll back to the previous binary or image.
- Only one update runs at a time. A second run stops with `Another update is already running (PID N)`.
- It never touches the keys, `node-info.yaml`, the BLS passphrase, the chain configuration, or the listener addresses.

The script does not update existing installs: replace the binary yourself and restart the service. After any update, run `sudo bash ~/telcoin-node-scripts/check-node.sh`.

5.3 Change the configuration

```
sudo bash ~/telcoin-node-scripts/edit-config.sh
```

The menu covers listener addresses, the metrics address, log verbosity, the BLS passphrase file, P2P ports, the Docker image, refreshing the chain configs, and restarting the node. Before any change it backs up the launch file to `<file>.bak.<timestamp>`. Changes take effect on the next restart.

Two cautions:

- For public RPC, use `install-caddy.sh` (see [Public RPC \(https and wss\) \(page 19\)](#)), not the menu's "RPC access" item. reth must stay on `127.0.0.1`.
- Change the P2P ports only to fix a conflict. The new ports must be open in every firewall layer and must match the multiaddrs in `node-info.yaml`.

5.4 Service commands

```
sudo systemctl start telcoin
sudo systemctl stop telcoin
sudo systemctl restart telcoin
journalctl -u telcoin -f
sudo tail -f /var/log/telcoin/telcoin.log
sudo systemctl reset-failed telcoin # after too many failed restarts
```

The README has the full [command reference](#).

5.5 Node Manager UI

The optional web UI shows health, logs and configuration, and can run updates. Install it on the node:

```
sudo bash ~/telcoin-node-scripts/ui/install-ui.sh
```

It listens on `127.0.0.1:8080` only. Reach it from your own machine through an SSH tunnel. Use either the helper script from a local copy of the deployment repository, or a tunnel by hand:

```
./open-ui.sh user@SERVER_IP
ssh -L 8080:localhost:8080 user@SERVER_IP # then open http://localhost:8080
```

To reach the dashboard on a public HTTPS name, run `sudo bash ~/telcoin-node-scripts/install-caddy.sh` and choose `[1] Dashboard`. Caddy then serves it behind a login. Public access is read-only; management stays on the tunnel. The dashboard needs its own hostname, such as `dashboard.<your-node-domain>`, separate from the public RPC name. See the README's [web UI](#) and [external dashboard access](#) sections.

5.6 Testnet add-ons

You can turn on the health monitor, log shipping, metrics shipping and VPN admin SSH after setup:

```
sudo bash ~/telcoin-node-scripts/setup-observability.sh # health, logs, metrics
sudo bash ~/telcoin-node-scripts/setup-vpn.sh           # VPN admin SSH
```

Read the [testnet add-ons guide](#) for what each one shares and how to turn it off. Read the [WireGuard VPN guide](#) before you enrol in the VPN.

5.7 Remove a node

If the node serves public RPC, turn that off first, because `remove-node.sh` does not touch Caddy:

```
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-disable
sudo bash ~/telcoin-node-scripts/remove-node.sh
```

The removal script asks before each component: service, Docker container and image, chain data, keys, binary, source tree, logs, service user, and the UI. Deleting the keys needs you to type `DELETE`. Do not delete the keys of a staked validator unless you hold a tested backup. Menu option `2) Wipe chain data only` keeps the keys and configuration and makes the node resync from scratch.

6 Public RPC (https and wss)

Public RPC serves your node's JSON-RPC at `https://<domain>/` and its WebSocket at `wss://<domain>/`. It also advertises both URLs in `node-info.yaml` so wallets and gateways can discover the node. reth itself stays on `127.0.0.1`. Caddy holds the TLS certificate on port 443 and proxies to reth on loopback.

Turning it on is a deliberate choice. The Caddy site has no authentication and no rate limiting, so anyone can send requests to it. It suits observers that are meant to serve the public.

IMPORTANT

Other nodes forward transactions to the RPC URLs the committee advertises. A validator therefore has to advertise a reachable RPC URL to receive forwarded transactions. Choose that URL deliberately and protect the endpoint behind it. The Caddy site has no rate limiting of its own, so put rate limits in front of it, keep reth on `127.0.0.1` behind Caddy, and allow nothing else through the firewall.

The README section on the [public RPC endpoint](#) covers the internals. The steps follow.

6.1 DNS first

Create an A record for the node's public name that points at the server's inbound public IP. Wait until it resolves before you enable anything. Caddy requests a Let's Encrypt certificate as soon as the site loads. If the name does not point at the server yet, issuance fails and repeated attempts get rate-limited.

- Allow inbound TCP 443 (required) and TCP 80 (recommended) in every firewall layer, and forward both on the router if the server is behind NAT.
- Behind NAT, or on a server with several addresses, pass the inbound IP with `--public-ip <ip>`. The address the scripts detect is the outbound one, which can differ.
- The RPC name and the dashboard name must be different, for example `node7.example.com` for RPC and `dashboard.node7.example.com` for the dashboard.

6.2 New install

```
DOMAIN=node7.example.com # your node's public name
sudo bash ~/telcoin-node-scripts/setup-node.sh --rpc-domain "$DOMAIN"
sudo bash ~/telcoin-node-scripts/setup-node.sh --rpc-domain "$DOMAIN" \
  --public-ip 203.0.113.10 # behind NAT
```

Or answer 2) Public at the RPC access prompt. With a domain, key generation writes

`https://<domain>/` and `wss://<domain>/` into `node-info.yaml` when the release's keytool supports advertising RPC (`--rpc-http`, present since `v0.12.0-adiiri`). After the node starts, setup checks DNS and runs `install-caddy.sh --phase=rpc-enable`. That run does not restart the node when `node-info.yaml` already holds those URLs. If DNS is not ready, setup finishes anyway and prints the command to run later.

`--no-public-rpc` keeps RPC private without asking. It cannot be combined with `--rpc-domain` or `--rpc-public`; setup stops with an error (an `error` event in `--json` mode). `--rpc-public` without `--rpc-domain` leaves RPC private and prints a warning with the command to enable it later.

6.3 Existing node

Check DNS first, then enable:

```
DOMAIN=node7.example.com # your node's public name
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-check-dns \
  --rpc-domain "$DOMAIN"
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-enable \
  --rpc-domain "$DOMAIN"
```

`rpc-check-dns` changes nothing and exits 0 only when the name reaches this server. `rpc-enable` writes the Caddy site, opens 80 and 443 in `ufw` when `ufw` is active, and makes sure reth serves WebSocket. It then sets the advertised URLs in `node-info.yaml` and restarts the node once if anything there changed. Add `--public-ip <ip>` behind NAT. If the dashboard already uses this name, create the A record for `dashboard.<domain>` and add `--move-dashboard-to dashboard.<domain>` to move it in the same run. For a guided run, start `sudo bash ~/telcoin-node-scripts/install-caddy.sh` and choose [2] Public RPC endpoint.

`rpc-enable` does not write any keys to `.node-meta`.

6.4 Verify

```
curl -s https://$DOMAIN/ -H 'content-type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"eth_chainId","params":[]}'
```

On testnet the answer is `"result":"0x7e1"` (chain ID 2017). The first request after enabling can take a few seconds while the certificate is issued.

For WebSocket, connect with `websocat wss://$DOMAIN/` or `wscat -c wss://$DOMAIN/` and send the same JSON request.

```
sudo bash ~/telcoin-node-scripts/check-node.sh
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-status
```

The health check has a public RPC block that probes https and wss through Caddy and shows what `node-info.yaml` advertises. It ends with `public RPC: OK`, or with `public RPC: WARN -- <reasons>` plus the command that fixes it. Run it with `sudo`; without root it can only report `public RPC: unknown` (`.node-meta` not readable – run with `sudo`). `--phase=rpc-status` shows whether the site is enabled, what is advertised, and whether reth's WebSocket port is listening.

6.5 Turn it off

```
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-disable
```

This clears the advertised URLs from `node-info.yaml`, restarts the node and removes the Caddy site. Once no Caddy site remains, it also closes 80 and 443 in `ufw`.

7 Validators: stake and activate

The stake and activation calls go to the ConsensusRegistry contract at `0x07E17e17E17e17E17e17E17E17E17e17E17e17E17e17E17e17E17e1`, the same address on testnet and devnet. The BLS keys never leave the node. You export the stake calldata on the node, then sign and send the transaction from the machine that holds your wallet. Use the commands below; they match the deployed contracts.

7.1 Prerequisites

- Association approval. Only GSMA-approved mobile network operators can validate; start with support@telcoin.org.
- Hardware at the validator tier (see [Hardware per role \(page 7\)](#)).
- UDP 49590 and 49594 reachable from the internet (see [Ports \(page 8\)](#)).
- A synced node (see [Confirm the node is syncing, then synced \(page 14\)](#)).
- A publicly routable RPC URL advertised in `node-info.yaml` (see [Public RPC \(https and wss\) \(page 19\)](#)). Other nodes forward user transactions to the committee's advertised URLs and skip private addresses, unless the chain's `parameters.yaml` sets `allow_private_forward_targets: true` (it defaults to `false`).
- Foundry's `cast` on the wallet machine, and the wallet holding your execution address.

7.2 Set variables

On the wallet machine:

```
REG=0x07E17e17E17e17E17e17E17E17E17e17E17e17E17e17E17e17E17e1
RPC=https://rpc.telcoin.network
ADDR=0xYOUR_EXECUTION_ADDRESS
```

7.3 Confirm the ConsensusNFT

```
cast call $REG "balanceOf(address)(uint256)" $ADDR --rpc-url $RPC
```

`1` means the Association has minted your ConsensusNFT and you can stake. `0` means approval is not complete yet.

7.4 Export the stake calldata on the node

The keytool reads only `node-info.yaml`; it needs neither the passphrase nor the private keys. For source builds (an existing install uses its own binary path):

```
sudo /opt/telcoin/telcoin-network keytool export-staking-args \
  --node-info /var/lib/telcoin/node-info.yaml --calldata
```

For Docker installs:

```
IMAGE=$(sudo sed -n 's/^DOCKER_IMAGE=//p' /etc/telcoin/.node-meta)
sudo docker run --rm -v /var/lib/telcoin:/home/nonroot/data:ro "$IMAGE" \
  telcoin keytool export-staking-args \
  --node-info /home/nonroot/data/node-info.yaml --calldata
```

`export-staking-args` ships with current releases. If the keytool does not recognise it, update the node first (see [Update the node \(page 16\)](#)).

Copy the `0x...` output to the wallet machine and check that it is calldata for the right function:

```
CALLDATA=0x... # paste the output here
[ "${CALLDATA:0:10}" = "$(cast sig 'stake(bytes,(bytes))')" ] && echo "calldata OK"
```

7.5 Read the stake amount

```
cast call $REG "getCurrentStakeConfig()(uint256,uint256,uint256,uint32)" \
  --rpc-url $RPC
STAKE=$(cast call $REG \
  "getCurrentStakeConfig()(uint256,uint256,uint256,uint32)" \
  --rpc-url $RPC | awk 'NR==1 {print $1}')
```

The four values are the stake amount in wei, the minimum reward withdrawal, the issuance per epoch, and the epoch length in blocks. `STAKE` holds the first one. Send exactly that amount, or the call reverts with `InvalidStakeAmount`. Read it just before you stake, because a new stake configuration takes effect at the next epoch. Once you have staked, the stake version recorded on your validator fixes the amount that applies to it. The stake version is the sixth field of `getValidator`, and `stakeConfig(uint8)` looks up the amount for it.

7.6 Stake

```
cast send $REG $CALLDATA --value $STAKE --from $ADDR --ledger --rpc-url $RPC
```

Use `--trezor` for a Trezor. `--interactive` asks for a raw private key; avoid it for a funded validator address.

7.7 Activate

Once the node is synced and the stake has landed (status `1`, Staked):

```
cast send $REG "activate()" --from $ADDR --ledger --rpc-url $RPC
```

Status moves to `2` (PendingActivation) and then to `3` (Active) at the next epoch boundary.

7.8 Watch your status

```
cast call $REG \
  "getValidator(address)(address,uint32,uint32,uint8,bool,uint8,uint8)" \
  $ADDR --rpc-url $RPC
```

The seven lines are the validator address, activation epoch, exit epoch, status, `isRetired`, stake version and region. The fourth line is the status (see [Status values \(page 24\)](#)). The same information, with the next step spelled out, comes from:

```
sudo bash ~/telcoin-node-scripts/check-node.sh --address $ADDR
```

Other signals:

- The Node Manager UI switches to the validator view once the node is synced and its on-chain status is 1 to 4.
- `tn_nodeMode` (see [Checking by hand \(page 14\)](#)) changes from `Observer` to `CvvActive` while the node holds a committee seat.
- The current epoch and its committee:

```
cast call $REG "getCurrentEpoch()(uint32)" --rpc-url $RPC
cast call $REG \
  "getCurrentEpochInfo()(address[],uint256,uint64,uint32,uint32,uint8)" \
  --rpc-url $RPC
```

The second call's output starts with the list of committee addresses.

7.9 When you get a committee seat

The committee for each epoch is fixed two epochs in advance. Expect your first possible seat at the third epoch boundary after `activate()`. On testnet's six-hour epochs that is 12 to 18 hours; the exact lead time is a protocol parameter. Selection is random only when there are more eligible validators than seats. Between seats an Active validator reports `Observer`, which is normal.

7.10 Status values

Status	Name	What it means	Your next step
(call reverts)	No record	No ConsensusNFT for this address.	Finish Association approval.
0	Undefined	NFT minted, not staked.	Stake (see Export the stake calldata on the node (page 21) onwards).
1	Staked	Stake received.	<code>activate()</code> once synced.
2	PendingActivation	Activation queued.	Wait for the next epoch boundary.
3	Active	Eligible for committee seats.	Keep the node healthy.
4	PendingExit	<code>beginExit()</code> called.	Wait until the protocol exits you.
5	Exited	Out of the validator set.	Wait one more epoch, then <code>unstake</code> (see Rewards and exit (page 24)).
6	Any	A reserved sentinel. On its own you should never see it.	Contact the Association.
6 with <code>isRetired true</code>	Retired	<code>unstake</code> ran: NFT burned, stake returned.	None. This identity cannot stake again.

`check-node.sh` prints `Status: Retired` for the last row and appends `(Retired)` to any other status that has `isRetired` set. The scripts treat statuses 1 to 4 as staked and everything else as not staked.

7.11 Rewards and exit

Claim accrued rewards without exiting once they reach `minWithdrawAmount`, the second value of the stake configuration:

```
cast send $REG "claimStakeRewards(address)" $ADDR --from $ADDR --ledger --rpc-url $RPC
```

To leave, call `beginExit()` while Active:

```
cast send $REG "beginExit()" --from $ADDR --ledger --rpc-url $RPC
```

It reverts if too few validators would remain. The status moves to PendingExit (4), and the protocol exits you once no current or upcoming committee needs you. When the status reads Exited (5), wait one more epoch, then reclaim the stake:

```
cast send $REG "unstake(address,bool)" $ADDR false --from $ADDR --ledger --rpc-url $RPC
```

WARNING

`unstake` permanently retires the validator and burns its ConsensusNFT. The address cannot be reused, and validating again needs new keys and a new NFT.

The second argument, `acceptRewardShortfall`, stays `false`; `true` forfeits any rewards the Issuance contract cannot cover. `unstake` also works while the status is still Staked (1), before activation.

7.12 What not to do

- Never regenerate keys for a staked identity, and never pass `--force` to the keytool for one. The registered BLS key would no longer match the node.
- Never run the same keys on two servers at once.
- When you re-run `setup-node.sh` on a staked node, answer No to `Overwrite existing keys?`.

8 Automation

8.1 Setup without prompts

`setup-node.sh --json` runs in two phases so you can back up the keys in between. Pass the same flags to both phases:

```
read -rs TN_BLS_PASSPHRASE && export TN_BLS_PASSPHRASE
IMAGE=us-docker.pkg.dev/telcoin-network/tn-public/adiri:v0.15.0-adiri # your tag
ARGS=(--network testnet --install-method docker --docker-image "$IMAGE"
  --address 0xYOUR_EXECUTION_ADDRESS
  --external-primary /ip4/203.0.113.10/udp/49590/quic-v1
  --external-worker /ip4/203.0.113.10/udp/49594/quic-v1
  --listener-primary /ip4/10.0.0.5/udp/49590/quic-v1
  --listener-worker /ip4/10.0.0.5/udp/49594/quic-v1)
sudo --preserve-env=TN_BLS_PASSPHRASE bash ~/telcoin-node-scripts/setup-node.sh \
  --json --phase=keygen "${ARGS[@]}"
```

Back up `node-keys/` and `node-info.yaml` now (see [Back up the keys now \(page 13\)](#)), then run the second phase in the same shell:

```
sudo bash ~/telcoin-node-scripts/setup-node.sh \
  --json --phase=finalize "${ARGS[@]}"
```

- `keygen` runs preflight, installs dependencies, creates the service user and generates the keys. It writes no unit and starts nothing, and it refuses to run if `node-keys/` already exists.
- `finalize` writes the configuration, creates and starts the service, and enables public RPC when you pass `--rpc-domain`.
- The passphrase comes only from `TN_BLS_PASSPHRASE`; keep it off the command line.
- Each phase prints one JSON object per line on stdout, with `event` set to `step`, `log`, `error` or `done`. The last line is `{"event": "done", "ok": true|false, ...}`. Human-readable output goes to stderr.
- `--network` takes `testnet` (or `adiri`) or `devnet`. Mainnet is coming.
- `--install-method` takes `source` (with `--build-ref <tag>`) or `docker` (with `--docker-image <ref>` on both phases). `--json` mode does not detect the image, and without the flag `finalize` would write a launch file with no image.
- `existing` is not reliable with `--json`. There is no flag for the binary path and `finalize` assumes `/opt/telcoin/telcoin-network`, so install the binary there first or use the interactive setup.
- Other flags: `--data-dir`, `--passphrase-method loadcredential|tpm`, `--public-ip`, `--advertised-name`, `--service-user`, `--service-group`, `--genesis-dir`, `--enable-healthcheck-monitor`, and the public RPC flags from [Public RPC \(https and wss\) \(page 19\)](#).

8.2 Advertised RPC URL flags

`--rpc-domain <d>` fills in all four of these, so most installs never pass them:

Flag	Written to	Derived from <code>--rpc-domain <d></code>
<code>--rpc-http <url></code>	<code>node-info.yaml</code> , published to the network	<code>https://<d>/</code>
<code>--rpc-ws <url></code>	<code>node-info.yaml</code> ; needs <code>--rpc-http</code> or <code>--rpc-domain</code> , otherwise it is ignored with a warning	<code>wss://<d>/</code>
<code>--public-rpc-url <url></code>	<code>.node-meta PUBLIC_RPC_URL</code> , for the UI and tooling only	<code>https://<d></code>
<code>--public-ws-url <url></code>	<code>.node-meta PUBLIC_WS_URL</code>	<code>wss://<d></code>

An explicit flag wins and is used as given. Derived URLs reach key generation only when the keytool supports `--rpc-http`; otherwise they are written when `rpc-enable` runs. If an explicit URL differs from the domain, setup warns that `rpc-enable` will replace it.

8.3 Other scripts

Script	Non-interactive forms
<code>install-caddy.sh</code>	<code>--phase=rpc-status</code> , <code>rpc-check-dns</code> , <code>rpc-enable</code> , <code>rpc-disable</code> and the dashboard phases <code>status</code> , <code>check-dns</code> , <code>enable</code> , <code>disable</code> , with or without <code>--json</code> . The dashboard password comes from <code>TN_CADDY_PASSWORD</code> .
<code>update-node.sh</code>	<code>--json --check</code> , <code>--json --prepare --ref <ref></code> , <code>--json --apply --yes</code> , <code>--json --discard</code> . A failed apply rolls back; its done line carries <code>"ok":false</code> and a boolean <code>rolled_back</code> (true when the rollback succeeded).
<code>firewall-setup.sh</code>	<code>--reset</code> , <code>--json --status</code> , <code>--json --enable</code> , <code>--json --port <49590/udp 49594/udp 43174/tcp> <on off></code> .
<code>edit-config.sh</code>	<code>--json --set <field>=<value></code> for <code>primary_listener</code> , <code>worker_listener</code> , <code>metrics</code> , <code>verbosity</code> or <code>docker_image</code> .
<code>remove-node.sh</code>	<code>--json --remove observer validator --scope service data keys --yes [--remove-ui]</code> . The <code>--remove</code> value is a UI slot name, not a role; either one removes the node.

The status and DNS-check phases and `update-node.sh --json --check` print a single JSON object instead of an event stream. `TN_ASSUME_YES=true` answers yes to the scripts' yes/no prompts. `check-node.sh` has no JSON mode and always exits 0, so parse its verdict line.

9 Troubleshooting

Start with `sudo bash ~/telcoin-node-scripts/check-node.sh` and `journalctl -u telcoin -n 200`. The tables below group the common symptoms by area.

9.1 Sync and node mode

Symptom	Likely cause	Fix
<code>tn_nodeMode</code> returns <code>Observer</code> while <code>getValidator</code> shows Active	Not seated yet, or between seats.	Normal. See When you get a committee seat (page 23) .
<code>tn_nodeMode</code> stays at <code>CvvInactive</code>	A committee member that has not caught up.	Treat it like a lagging node (next row).
<code>CATCHING UP</code> and the lag does not shrink between runs	Peers cannot reach the node, the disk is full or slow, or the node keeps restarting.	Check UDP 49590 and 49594 in every firewall layer, the public IP in the external multiaddrs in <code>node-info.yaml</code> , disk space, and the journal for restarts.
<code>eth_syncing</code> returns <code>false</code> while the node is behind	Expected on Telcoin Network.	Compare <code>eth_blockNumber</code> instead (see Checking by hand (page 14)).

9.2 Setup and preflight

Symptom	Likely cause	Fix
Preflight prints <code>[WARN] <role>: below minimum</code>	The server is under that role's minimum.	A warning only. Remember the check counts logical CPUs.
<code>systemd <version> detected -- version 247+ required.</code>	The OS is too old.	Move to Ubuntu 22.04+, Debian 12+ or RHEL 9+.
<code>CVE-2026-31431 (Copy Fail) -- setup cannot continue</code>	The <code>algif_aead</code> module is not blocked, or is still loaded.	Apply the mitigation from https://copy.fail , unload the module or reboot, then re-run setup.
Port <code><port>/<proto></code> (...) is already in use.	Another process holds the port.	Find it with <code>sudo ss -ltnp</code> (UDP) or <code>sudo ss -ltnp</code> (TCP), then stop it or choose another port.

9.3 Public RPC and certificates

Symptom	Likely cause	Fix
<code><domain></code> already serves the Node Manager dashboard -- the dashboard and the public RPC endpoint need DIFFERENT hostnames	RPC and dashboard on one name.	Create dashboard.<domain> in DNS, then add <code>--move-dashboard-to dashboard.<domain></code> to <code>rpc-enable</code> .
<code>rpc-check-dns</code> fails, or setup says Public RPC not enabled yet	The A record is missing, not propagated, or points at the outbound IP of a NAT host.	Fix the record, pass <code>--public-ip <inbound-ip></code> behind NAT, then re-run <code>rpc-enable</code> .
The certificate is not issued, or Let's Encrypt reports a rate limit	DNS or ports 80 and 443 were wrong when Caddy asked.	Fix DNS and the ports first, then wait out the rate limit; repeated attempts extend it.
port 80 is in use by 'apache2' (or 443, or nginx)	Another web server holds the ports Caddy needs.	<code>sudo systemctl disable --now apache2</code> (or nginx), or run <code>sudo bash ~/telcoin-node-scripts/install-caddy.sh</code> and let it guide you.
public RPC: unknown (.node-meta not readable – run with sudo)	<code>check-node.sh</code> ran without root.	Run it with <code>sudo</code> .
<code>wss://<domain>/</code> returns 502	Nothing listens on the WebSocket port 8546 yet.	Wait if the node is still starting. Otherwise check <code>install-caddy.sh --phase=rpc-status</code> , and re-run <code>rpc-enable</code> if WebSocket is not listening.

9.4 Service and keys

Symptom	Likely cause	Fix
The node cannot decrypt its BLS key at start	<code>/etc/telcoin/bls-passphrase</code> does not hold the passphrase the keys were generated with, or a TPM reset or rebuild stopped a TPM install from unsealing.	Restore the correct passphrase. <code>edit-config.sh</code> option 5 rewrites the file but never re-encrypts the keys. A TPM install needs the passphrase you stored offline (TPM notes).
<code>systemctl status telcoin</code> shows <code>start-limit-hit</code>	Five failed starts within 60 seconds.	Fix the cause from the journal, then <code>sudo systemctl reset-failed telcoin</code> && <code>sudo systemctl start telcoin</code> .

9.5 Staking and exit

Symptom	Likely cause	Fix
<code>stake</code> reverts with <code>RequiresConsensusNFT</code>	No ConsensusNFT for the sending address.	Check Confirm the ConsensusNFT (page 21) , and that <code>--from</code> is your execution address.
<code>stake</code> reverts with <code>InvalidStakeAmount</code>	<code>--value</code> differs from the stake amount.	Re-read the amount (see Read the stake amount (page 22)).
<code>stake</code> reverts with <code>InvalidProofOfPossession</code>	The calldata was signed for a different execution address or node.	Re-export on this node and send from the address you entered at key generation. If the address has to change, contact the Association first.
<code>stake</code> or <code>activate</code> reverts with <code>InvalidStatus</code>	The validator is not in the status the call needs.	Check the status (see Watch your status (page 23)).
<code>stake</code> reverts with <code>DuplicateBLSPubkey</code>	This BLS key is already registered.	Contact the Association. Do not regenerate keys.
<code>beginExit</code> reverts	The validator is not Active yet, or too few validators would remain.	Wait, then retry.
<code>unstake</code> reverts with <code>IneligibleUnstake</code>	The validator is neither Staked nor one epoch past Exited.	Wait for Exited plus one epoch.

10 Getting help

Email support@telcoin.org for everything, including validator onboarding, approval and hardware.

Include:

- The full output of `sudo bash ~/telcoin-node-scripts/check-node.sh`. Validators add `--address <your address>`.
- The last 200 log lines: `journalctl -u telcoin -n 200 --no-pager`.
- The node version (the Docker image tag, or the tag you built) and the script versions that `update-scripts.sh` lists.
- The network (testnet or devnet) and your install method.

WARNING

Never send the contents of `node-keys/` or your BLS passphrase, to support or to anyone else.